AI DRIVEN SOC MONITORING ON ELASTIC

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF VOCATIONAL
(CYBER SECURITY AND FORENSICS)

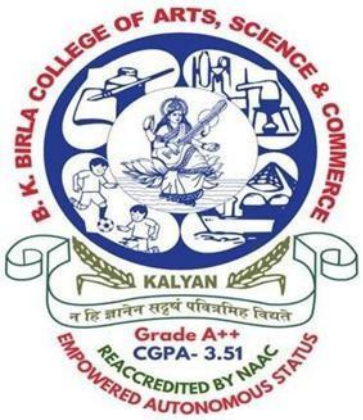
By

Nayan Patel & Omar Khan

Under the esteemed guidance of

Mr. Akash Kamble

Coordinator, Cyber Security and Forensics
Assistant Professor, Dept. of IT



DEPARTMENT OF INFORMATION TECHNOLOGY

(Affiliated to University of Mumbai)

B.K. Birla College of Arts, Science and Commerce (Empowered Autonomous), Kalyan (W)

Academic Year: 2025–2026

B. K. BIRLA COLLEGE OF ARTS, SCIENCE AND COMMERCE

(Empowered Autonomous Status)

(Affiliated to University of Mumbai)

KALYAN, MAHARASHTRA-421301

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, **"AI DRIVEN SOC MONITORING ON ELASTIC"**, is submitted by **Mr. Nayan Patel** bearing **Seat. No: 4559487** submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF VOCATION in CYBER SECURITY AND FORENSICS from University of Mumbai, is a bonified work carried out during academic year 2025-2026.

Internal Guide	External Examiner	Coordinator

Date: _____ **College seal:** _____

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN NO.:2023016400755217 Roll no.: 34

- Name of the Student:
  Mr. Nayan Patel
- Title of the Project:**AI DRIVEN SOC MONITORING ON ELASTIC**
- Name of the Guide:
  Mr. Akash Kamble
- Teaching experience of the Guide
- Is this your first submission? Yes ☐ No ☒

Signature of the student

Date: ……

Signature of the Guide

Date: ……

Signature of the Coordinator

Date: ……

PREFACE

The field of cybersecurity has evolved dramatically over the past decade, with threats becoming increasingly sophisticated, frequent, and difficult to detect using traditional rule-based approaches. As organization generate millions of log events daily, the need for intelligent, automated, and real-time monitoring systems has never been more critical.

This project, **AI Driven SOC Monitoring on Elastic**, was conceived with the goal of building a practical, deployable Security Operations Centre monitoring solution using entirely open-source tools. Rather than simulating a security environment, this system was built and tested against real Windows event logs collected from a live host machine, resulting in over 1.6 million indexed documents and genuine threat detection's.

The motivation behind this project stems from a simple observation — most academic cybersecurity projects demonstrate concepts in isolation. A phishing classifier here, a log parser there. This project attempts to bring multiple disciplines together into a single functioning pipeline: log collection, data indexing, statistical analysis, risk scoring, anomaly detection, alerting, and visualization — all working in concert as a real SOC would operate.

Throughout the development of this system, we gained hands-on exposure to enterprise-grade tools including the Elastic Stack, Windows Security event logging, Python-based data engineering, and statistical behavioral modelling. The experience of seeing a real user account flagged as ANOMALOUS with a risk score of 702 — not in a simulation, but from actual system activity — reinforced the practical value of what was built.

We hope this project serves as a useful reference for students and practitioners interested in building lightweight SIEM solutions using open-source technologies, and demonstrates that meaningful cybersecurity tooling does not require expensive enterprise licence — only curiosity, persistence, and the right stack.

Acknowledgement

We would like to express our sincere gratitude to all those who supported and guided us throughout the development of this project.

First and foremost, we are deeply thankful to our project guide, Mr. Akash Kamble, for his invaluable guidance, encouragement, and technical support throughout the project. His expertise in cybersecurity, SIEM systems, and log analysis helped shape every stage of this work.

We extend our heartfelt thanks to the Principal and the Head of the Department of Cyber Security & Forensics, B.K. Birla College, for providing the necessary resources and a conducive academic environment.

We would also like to acknowledge the open-source community and the developers behind the tools and libraries used in this project, including the Elastic team (Elasticsearch, Kibana, Winlogbeat), the Python Software Foundation, and all contributors to the elasticsearch-py library.

Finally, we thank our families and friends for their patience, encouragement, and unwavering moral support throughout the entire project.

Declaration

I declare that this submission represents my ideas in my own words and where others ideas or words have been declare that I have adequately cited and referenced the original sources. I also declare that I have adhered to all the principles of academic honesty and integrity and have not misrepresented or fabricated any idea/data/fact/source in my submission.

The project is done in partial fulfillment of the requirements for the award of degree of BACHELOR OF VOCATIONAL (CYBERSECURITY AND FORENSICS) to be submitted as final semester project as part of our curriculum.

Name and Signature of the Student

Table of Contents

Chapter 1: Introduction

1.1 Background & Motivation

Modern enterprises face an unprecedented volume of cyber threats including brute force attacks, insider threats, unauthorised access, and lateral movement by adversaries. Security Operations Centres (SOCs) are the first line of defence, responsible for monitoring, detecting, and responding to incidents in real time. However, manual monitoring of thousands of log events per second is infeasible without automated intelligence.

The Elastic Stack (formerly ELK Stack) has emerged as a leading open-source platform for log collection, indexing, and visualisation at scale. Combining Winlogbeat for Windows event log shipping, Elasticsearch for scalable full-text indexing, and Kibana for interactive dashboards, it provides a solid foundation for a lightweight SIEM solution.

This project was motivated by the need to extend the Elastic Stack beyond passive log storage by adding an active Python-based intelligence layer that computes per-user risk scores, detects behavioural anomalies, and feeds enriched threat data back into Elasticsearch for unified dashboarding.

Modern cybersecurity landscapes are continuously evolving due to the rapid digitization of infrastructure, cloud adoption, and remote work environments. Organizations today generate **massive volumes of security logs**, often exceeding millions of events per day. These logs contain critical information about authentication attempts, system behavior, network activity, and process execution.

Traditional Security Operations Centres (SOCs) relied heavily on manual log inspection and rule-based detection mechanisms. However, these approaches are no longer sufficient due to:

- High volume of log data
- Sophisticated attack techniques (low-and-slow attacks)
- Insider threats that mimic normal behavior
- Increasing false positives in rule-based systems

To address these challenges, modern SOCs are shifting toward **automation and intelligent analysis**. This project introduces an **AI-driven approach using statistical methods** to enhance detection capabilities.

By integrating **Elastic Stack with Python-based analytics**, this system transforms raw logs into meaningful security insights, enabling proactive threat detection.

1.2 Problem Statement

Traditional log management systems suffer from the following limitations:

- Raw log ingestion tools like Winlogbeat provide no built-in risk classification — they store events but do not score users.
- Kibana dashboards display raw counts but cannot natively compute composite risk indicators based on multiple weighted event types.
- Static threshold-based alerts miss sophisticated low-and-slow attacks that spread activity across time.
- There is no out-of-the-box mechanism to establish a dynamic behavioural baseline per user and flag statistical outliers automatically.

This project addresses all four limitations by building a Python risk engine that queries Elasticsearch in real time, computes weighted multi-factor risk scores, establishes a dynamic statistical baseline, and writes enriched risk documents back to a dedicated Elasticsearch index for unified monitoring.

Despite advancements in SIEM tools, several limitations persist in traditional systems:

1. **Lack of Contextual Intelligence**
   Logs are stored but not interpreted intelligently. Security teams must manually analyze patterns.
2. **Static Detection Mechanisms**
   Fixed thresholds (e.g., 5 failed logins) fail to detect:
   1. Distributed brute-force attacks
   2. Slow credential stuffing
3. **No Behavioral Baseline**
   Systems do not understand what is “normal” for a user.
4. **High False Positives**
   Alerts are triggered without context, leading to alert fatigue.
5. **Limited Correlation Across Events**
   Individual events are analyzed separately rather than as part of a sequence.

This project solves these issues by introducing:

- Multi-factor risk scoring
- Statistical anomaly detection
- User-based behavioral profiling

1.3 Objectives

The primary objectives of this project are:

1. Deploy Winlogbeat on a Windows host to continuously ship Windows Security event logs to Elasticsearch over HTTPS.
2. Build a Python risk engine that queries Elasticsearch for four key event types (Event IDs 4625, 4624, 3, and 1) and aggregates per-user event counts.
3. Compute a weighted composite risk score: risk_score = (3×F) + (1×S) + (2×N) + (4×P) and classify into LOW, MEDIUM, HIGH threat levels.
4. Establish a statistical behaviour baseline and flag users as ANOMALOUS when activity exceeds mean + 1.5×std or risk_score exceeds 200.
5. Write enriched risk documents back to a user_risk_index in Elasticsearch for persistent audit trails.
6. Build a Kibana dashboard with six visualisations covering user risk, threat classification, login timelines, network and process activity.
7. Implement an Elastic Observability brute force attack detection alert rule triggering when failed logins exceed a custom threshold within a 1-minute window.

1.4 Scope of the Project

The AI Driven SOC Monitoring System is scoped to monitor a single Windows host machine through Winlogbeat log shipping. The scope includes:

- Real-time Windows event log collection and shipping using Winlogbeat 9.x to Elasticsearch 8.x.
- Multi-factor user risk scoring from four event categories: failed logins, successful logins, network connections, and process creations.
- Statistical behavioural anomaly detection using population mean and standard deviation.
- A Kibana dashboard suite with six purpose-built visualisations.
- A brute force attack alert rule using Elastic Observability with custom threshold conditions.
- Write-back of enriched risk scores to a dedicated Elasticsearch index (user_risk_index).

Out of scope: Multi-host agent deployment, deep packet inspection, email/SOAR integration, and threat intelligence feed enrichment are identified as future enhancements.

The system is designed as a **lightweight SIEM prototype**, focusing on practical implementation rather than enterprise-scale deployment.

Included Scope:

- Real-time log ingestion from Windows host
- Event-based risk scoring
- Statistical anomaly detection
- Visualization via Kibana dashboards
- Alert generation for brute-force attacks

Limitations:

- Single-host monitoring
- No external threat intelligence integration
- No automated incident response

Practical Use Case:

This system can be used in:

- Small organizations
- Academic labs
- Cybersecurity training environments

1.5 Project Methodology

The project follows a 5-phase development methodology:

8. Infrastructure Setup: Install and configure Elasticsearch and Kibana on Ubuntu 24.04 WSL. Install Winlogbeat on Windows host and configure winlogbeat.yml for HTTPS log shipping.
9. Log Collection & Indexing: Run Winlogbeat to begin shipping Windows event logs. Verify data appears in Kibana Discover under the winlogbeat-* index pattern.
10. Risk Engine Development: Implement risk_engine.py on Ubuntu to query Elasticsearch, aggregate per-user event counts, compute weighted risk scores, establish statistical baseline, and classify threat levels.
11. Dashboard Development: Build six Kibana visualisations and assemble them into the AI driven SOC dashboard. Configure brute force alert rule in Elastic Observability.
12. Testing & Validation: Test all risk classifications, anomaly detection thresholds, brute force alert triggering, and dashboard real-time refresh.

The methodology follows a structured pipeline:

1. Data Collection Phase

- Windows logs are captured using Winlogbeat
- Logs include authentication, process, and network events

2. Data Storage Phase

- Logs are indexed into Elasticsearch
- Data is stored in structured JSON format

3. Data Processing Phase

- Python engine retrieves logs
- Aggregates data per user
- Applies statistical analysis

4. Detection Phase

- Risk scores calculated
- Behavioral anomalies identified

5. Visualization Phase

- Dashboards provide insights
- Alerts notify suspicious activity

Chapter 2: Literature Review

2.1 Introduction to Literature Review

The rapid evolution of cyber threats has significantly increased the demand for advanced Security Operations Centres (SOCs) capable of detecting and responding to attacks in real time. Traditional security monitoring systems primarily rely on rule-based detection mechanisms, which are often insufficient in identifying modern, sophisticated threats such as insider attacks, advanced persistent threats (APTs), and low-and-slow attacks.

This literature review explores existing research, technologies, and methodologies related to log monitoring, Security Information and Event Management (SIEM) systems, anomaly detection, and AI-driven cybersecurity solutions. The aim is to understand the limitations of traditional approaches and highlight the importance of intelligent, data-driven SOC monitoring systems.

2.2 Traditional Security Operations Centres (SOC)

A Security Operations Centre (SOC) is a centralized unit responsible for monitoring, detecting, analyzing, and responding to cybersecurity incidents. Traditional SOCs depend heavily on human analysts and predefined rules to identify suspicious activities.

In conventional systems, logs generated from various sources such as servers, applications, and network devices are collected and analyzed manually or through rule-based SIEM tools. These rules are typically based on known attack patterns, such as multiple failed login attempts or access from unusual locations.

However, traditional SOCs face several challenges:

- High volume of log data making manual analysis difficult
- Dependency on predefined rules, which cannot detect unknown threats
- Increased false positives leading to alert fatigue
- Lack of behavioral analysis capabilities

Due to these limitations, there is a growing need for automated and intelligent SOC systems that can adapt to evolving threats.

2.3 Security Information and Event Management (SIEM) Systems

SIEM systems play a crucial role in modern cybersecurity by collecting, storing, and analyzing log data from multiple sources. Popular SIEM tools include:

- Splunk
- IBM QRadar
- ArcSight
- Wazuh
- Elastic Stack (ELK)

These systems provide features such as:

- Centralized log management
- Real-time monitoring
- Alert generation
- Dashboard visualization

Despite their advantages, traditional SIEM systems have certain limitations:

- High cost (especially enterprise tools like Splunk and QRadar)
- Limited anomaly detection capabilities without additional modules
- Dependence on rule-based correlation
- Complex configuration and maintenance

Recent studies suggest that SIEM systems must integrate machine learning and statistical analysis to improve detection accuracy and reduce false positives.

2.4 Log Management and Analysis

Log management is the backbone of any SOC system. Logs provide detailed records of system activities, including user authentication, process execution, and network communication.

Tools like Elasticsearch and Logstash enable efficient storage and processing of large volumes of log data. These tools support:

- Real-time indexing of logs
- Fast search and query capabilities
- Structured data storage in JSON format

However, raw log data alone does not provide meaningful insights. Without proper analysis, logs remain underutilized. This has led to the development of advanced log analysis techniques that focus on extracting actionable intelligence from data.

2.5 Rule-Based Detection vs Anomaly Detection

Rule-Based Detection

Rule-based detection systems operate on predefined conditions. For example:

- Trigger an alert if failed login attempts exceed 5
- Detect access from blacklisted IP addresses

Advantages:

- Simple to implement
- Effective for known threats

Limitations:

- Cannot detect unknown or zero-day attacks
- Easily bypassed by attackers
- Generates high false positives

Anomaly Detection

Anomaly detection focuses on identifying deviations from normal behavior. Instead of relying on fixed rules, it builds a baseline of user or system activity and flags unusual patterns.

Techniques used include:

- Statistical analysis (mean, standard deviation)
- Machine learning models
- Behavioral profiling

Advantages:

- Detects unknown threats
- Reduces dependency on predefined rules
- Adapts to changing environments

Limitations:

- Requires proper baseline training
- May generate false positives if not tuned correctly

The shift from rule-based detection to anomaly detection represents a major advancement in SOC technologies.

2.6 AI and Machine Learning in Cybersecurity

Artificial Intelligence (AI) and Machine Learning (ML) are increasingly being used to enhance cybersecurity systems. These technologies enable automated threat detection, predictive analysis, and intelligent decision-making.

Applications of AI in SOC include:

- User behavior analytics (UBA)
- Threat prediction
- Automated incident response
- Malware detection

Machine learning models such as:

- Isolation Forest
- Neural Networks
- Clustering algorithms

are commonly used to detect anomalies in large datasets.

However, implementing full-scale AI models requires high computational resources and large training datasets. As a result, many practical systems use lightweight statistical approaches as an alternative.

2.7 Elastic Stack for SOC Monitoring

The Elastic Stack (Elasticsearch, Logstash, Kibana, and Beats) has emerged as a powerful open-source alternative to traditional SIEM systems.

Key components:

- **Elasticsearch**: Stores and indexes log data
- **Kibana**: Provides visualization and dashboards
- **Beats (Winlogbeat)**: Collects and ships logs
- **Logstash (optional)**: Processes and transforms data

Advantages:

- Open-source and cost-effective
- Highly scalable
- Real-time data processing
- Flexible and customizable

Limitations:

- Requires manual configuration
- Lacks built-in advanced analytics
- Needs additional scripting for intelligence

Due to its flexibility, Elastic Stack is widely used in academic and small-scale SOC implementations.

2.8 Existing Research and Gaps

Several research studies have explored SIEM systems, anomaly detection, and AI-driven security monitoring. While these systems provide valuable insights, they often suffer from:

- Lack of integration between log collection and analysis
- Over-reliance on either rule-based or ML-based approaches
- High implementation complexity
- Limited real-world deployment

There is a clear gap in systems that:

- Combine log collection, analysis, and visualization in one pipeline
- Use lightweight statistical methods for real-time detection
- Are cost-effective and easy to deploy

2.9 Summary of Literature Review

The literature review highlights that while traditional SOC and SIEM systems provide essential monitoring capabilities, they are not sufficient to handle modern cyber threats. The integration of intelligent analysis techniques such as anomaly detection and risk scoring is necessary to improve detection accuracy.

The Elastic Stack provides a strong foundation for building such systems, but it requires additional analytical layers to transform raw data into actionable insights. This project addresses these gaps by introducing a Python-based risk engine that enhances the capabilities of a traditional log monitoring system.

Chapter 3: Survey of Technologies

3.1 Elastic Stack Overview

The Elastic Stack is an open-source suite of tools designed for searching, analysing, and visualising data in real time. Originally known as the ELK Stack (Elasticsearch, Logstash, Kibana), it now also includes Beats (lightweight data shippers). In this project, the relevant components are Winlogbeat, Elasticsearch, and Kibana.

The Elastic Stack provides a **scalable and flexible architecture** for handling large volumes of log data.

Key Advantages:

- Real-time data processing
- Horizontal scalability
- Powerful search capabilities
- Open-source flexibility

Why Elastic Stack?

Compared to traditional SIEM tools:

- No licensing cost
- Highly customizable
- Strong community support

3.2 Winlogbeat 9.x

Winlogbeat is a lightweight, open-source Windows event log shipper developed by Elastic. It reads from the Windows Event Log API and ships structured JSON documents to Elasticsearch or Logstash. Key characteristics:

- Ships Windows Security, System, and Application event logs in real time.
- Supports HTTPS transport with configurable SSL settings.
- Produces 800+ indexed fields per document including @timestamp, event.code, user.name, host.name, and winlog.event_data fields.
- Configured via winlogbeat.yml specifying Elasticsearch output host, credentials, and log channels.
- Can be invoked from PowerShell using .\winlogbeat.exe -e for verbose debugging.

3.3 Elasticsearch 8.x

Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene. Key features utilised:

- Full-text search and KQL filtering across all ingested log fields.
- Index patterns (winlogbeat-*) enabling time-series queries across rolling indices.
- HTTPS API at https://localhost:9200 with basic authentication.
- Dedicated index (user_risk_index) to store enriched risk documents written back by the Python engine.
- Bool query DSL with range filters on @timestamp enabling time-windowed event aggregation.

3.4 Kibana 8.x

Kibana is the visualisation and dashboarding layer of the Elastic Stack. Features used:

- Visualize Library for creating and saving individual chart panels (bar charts, pie charts, line charts).
- Dashboard canvas for assembling multiple panels with shared KQL filters and time range controls.
- Discover module for raw log exploration with field filtering and field statistics.
- Observability Alerts module for creating custom threshold rules with configurable conditions.

3.5 Python Risk Engine Libraries

The Python risk engine (risk_engine.py) uses the following libraries:

- elasticsearch-py: The official Python client for Elasticsearch. Provides search() and index() operations.
- datetime / timedelta / UTC: Standard library modules for computing time windows (now - 1 hour).

- statistics: Standard library module providing mean() and stdev() for baseline computation.
- urllib3: Used to disable InsecureRequestWarning when verify_certs=False for development HTTPS.

3.6 Comparative Analysis

Feature	This Project	Splunk SIEM	IBM QRadar	Wazuh	ELK (vanilla)
Log Collection	Winlogbeat	Univ. Forwarder	WinCollect	Wazuh Agent	Beats
Risk Scoring	Custom Python	Proprietary	Built-in	Partial	None
Anomaly Detection	Statistical	ML (paid)	ML engine	Rule-based	None
Brute Force Alert	Yes	Yes	Yes	Yes	Manual
Custom Dashboard	Yes (Kibana)	Yes	Yes	Yes	Yes
Open Source	Yes	No (SaaS)	No (Enterprise)	Yes	Yes
Write-back Index	Yes	No	Yes	No	No

Chapter 4: Requirements & System Analysis

4.1 System Architecture Overview

The system is a pipeline architecture spanning four layers across two operating systems (Windows host + Ubuntu WSL). The data flows from Windows event logs through Winlogbeat into Elasticsearch, is processed by the Python risk engine, and is visualised in Kibana. Five functional layers:

- Collection Layer: Winlogbeat running on Windows host reads Windows Security event logs and ships JSON documents to Elasticsearch via HTTPS.
- Storage Layer: Elasticsearch indexes all incoming documents under winlogbeat-* time-series indices. The Python engine writes enriched risk documents to user_risk_index.
- Processing Layer: Python risk engine running on Ubuntu queries Elasticsearch, aggregates per-user event counts across a 1-hour sliding window, computes weighted risk scores, establishes statistical baseline, and classifies threat level and anomaly status.
- Alerting Layer: Elastic Observability custom threshold rule monitors winlogbeat-* for failed login counts exceeding a configurable threshold.
- Presentation Layer: Kibana dashboard with six visualisations providing real-time threat intelligence overview.

The system follows a **distributed pipeline model**, ensuring separation of concerns:

Benefits:

- Scalability
- Fault isolation
- Easy debugging

Cross-Platform Integration:

- Windows → Log generation
- Ubuntu WSL → Processing
- Elasticsearch → Central storage

4.2 Three-Layer Detection Pipeline

Every analysis cycle of the risk engine passes through four sequential processing stages:

13. Event Collection Stage: The get_event_counts(event_code) function queries winlogbeat-* for documents matching a specific event code within the last hour. Four collections are built: failed_logins (4625), successful_logins (4624), network_connections (3), process_creations (1).
14. Baseline Computation Stage: For each user, a composite baseline score (3×F)+(1×S)+(2×N)+(4×P) is computed. Population mean and std dev are calculated. Dynamic anomaly threshold = mean + 1.5 × std.
15. Risk Classification Stage: risk_score computed per user. Classification: LOW (≤10), MEDIUM (≤25), HIGH (>25). Activity exceeding threshold or risk_score ≥ 200 results in ANOMALOUS status.
16. Write-Back Stage: Enriched document with 9 fields is indexed into user_risk_index via es.index(

The detection pipeline is designed to simulate **real SOC workflows**:

1. Event ingestion
2. Event filtering
3. Data aggregation
4. Risk computation
5. Alert generation

This layered approach ensures:

- Accuracy
- Reduced false positives
- Better detection coverage

4.3 Hardware & Software Requirements

Component	Minimum	Recommended
Processor	Intel Core i3	Intel Core i5 or higher
RAM	8 GB DDR4	16 GB DDR4
Storage	5 GB SSD	20 GB SSD
OS (Host)	Windows 10	Windows 11
OS (Engine)	Ubuntu 22.04 WSL	Ubuntu 24.04 WSL

Category	Tool / Library	Version	Purpose
Log Collection	Winlogbeat	9.3.0	Windows event log shipping
Storage	Elasticsearch	8.x	Log indexing & search
Dashboard	Kibana	8.x	Visualisation & alerts
Runtime	Python	3.10+	Risk engine language
ES Client	elasticsearch-py	8.x	Elasticsearch Python client
Stats	statistics (stdlib)	built-in	Baseline calculation

Chapter 5: System Design

5.1 Introduction to System Design

System design is a critical phase in the development of any cybersecurity solution, as it defines how different components interact to achieve the desired functionality. The AI-driven SOC monitoring system is designed as a modular, scalable, and efficient pipeline that integrates log collection, storage, processing, and visualization.

The system follows a distributed architecture where each component performs a specific function. This modular approach ensures flexibility, easy debugging, and scalability for future enhancements. The design focuses on real-time monitoring, efficient data processing, and accurate threat detection.

5.2 Overall System Architecture

The system architecture consists of multiple layers working together to form a complete SOC monitoring pipeline. These layers include:

1. Data Collection Layer

This layer is responsible for collecting raw log data from the Windows host system. Winlogbeat is used to extract event logs such as authentication attempts, process execution, and network connections.

2. Data Storage Layer

All collected logs are stored in Elasticsearch. It acts as a centralized repository where data is indexed and made searchable in real time.

3. Processing Layer

The Python-based risk engine processes the collected data. It performs aggregation, risk scoring, and anomaly detection using statistical methods.

4. Alerting Layer

This layer monitors specific conditions such as failed login thresholds and generates alerts for potential attacks.

5. Visualization Layer

Kibana dashboards provide graphical representations of system activity, helping analysts quickly understand threats and trends.

This layered architecture ensures separation of concerns and improves system reliability.

5.3 Detailed Component Design

5.3.1 Winlogbeat Configuration Module

Winlogbeat acts as the data ingestion agent. It is configured using a YAML file where parameters such as log sources, output destination, and security settings are defined.

Key responsibilities:

- Capture Windows Security logs
- Convert logs into structured JSON format
- Securely transmit data to Elasticsearch

The configuration includes enabling specific event IDs and setting up HTTPS communication to ensure secure data transfer.

5.3.2 Elasticsearch Storage Module

Elasticsearch serves as the backbone of the system by storing and indexing all incoming data. It organizes logs into indices such as:

- winlogbeat-* (raw logs)
- user_risk_index (processed data)

Key features:

- Fast full-text search
- Scalable indexing
- Real-time data retrieval

It supports complex queries using Query DSL, allowing efficient filtering and aggregation of large datasets.

5.3.3 Python Risk Engine Module

The Python risk engine is the core intelligence component of the system. It performs the following tasks:

- Retrieves logs from Elasticsearch
- Aggregates data per user
- Computes weighted risk scores
- Detects anomalies using statistical methods
- Writes results back to Elasticsearch

This module transforms raw logs into meaningful security insights, enabling proactive threat detection.

5.3.4 Kibana Visualization Module

Kibana provides an interactive interface for monitoring and analysis. It includes:

- Dashboards
- Charts and graphs
- Real-time data updates

Visualizations help in identifying patterns such as:

- High-risk users
- Login trends
- Network activity

This module enhances decision-making by presenting complex data in an easy-to-understand format.

5.4 Data Flow Design

The system follows a structured data flow pipeline:

1. Windows system generates event logs
2. Winlogbeat collects and forwards logs
3. Elasticsearch stores and indexes data
4. Python engine retrieves and processes logs
5. Risk scores and anomalies are calculated
6. Processed data is written back to Elasticsearch
7. Kibana visualizes results

5.5 Application Architecture

The system follows a pipeline architecture with five loosely coupled components connected via Elasticsearch as the central data hub.

Module / File	Purpose
winlogbeat.yml	Winlogbeat configuration: event log channels, Elasticsearch output URL, credentials, SSL settings
winlogbeat.exe	Windows service collecting and shipping Security/System event logs to Elasticsearch
risk_engine.py	Python script: queries Elasticsearch, computes risk scores, detects anomalies, writes back to user_risk_index
winlogbeat-*	Elasticsearch time-series index receiving all Windows event log documents from Winlogbeat
user_risk_index	Elasticsearch index storing enriched risk documents with per-user threat classification
Kibana Dashboard	Six visualisation panels assembled into the AI driven SOC dashboard
Observability Alert	Brute force detection rule with custom threshold monitoring failed logins per minute

5.6 Data Flow Design

17. Windows host generates Security event logs (failed logins, successful logins, process creations, network connections).
18. Winlogbeat reads event logs via Windows Event Log API and ships structured JSON documents to Elasticsearch at https://localhost:9200 under the winlogbeat-* index.
19. Python risk engine on Ubuntu WSL connects to Elasticsearch and queries the last hour's events, filtered by event code.
20. Per-user event counts are aggregated for all four event types.
21. Behaviour baseline (mean + std) is computed from the population activity vector.
22. Per-user risk_score, threat_level, and behavior_status are computed and printed to the terminal.
23. Enriched risk document is written back to user_risk_index in Elasticsearch.
24. Kibana dashboard reads from both winlogbeat-* and user_risk_index to render six real-time panels.

5.7 Threat Scoring Engine

Component	Weight	Rationale
Failed Logins (F)	×3	Highest risk indicator — multiple failed logins suggest brute force or credential stuffing
Process Creations (P)	×4	Highest weight — unusual process spawning indicates malware execution or lateral movement
Network Connections (N)	×2	Medium risk — abnormal outbound connections suggest C2 or data exfiltration
Successful Logins (S)	×1	Lowest weight — normal activity but high counts relative to failed logins is suspicious
LOW threshold	≤10	Routine user activity, no action required
MEDIUM threshold	≤25	Elevated activity, monitor closely
HIGH threshold	>25	Serious threat, immediate investigation required

5.8 Anomaly Detection Design

- Baseline Vector: score_u = (3×F_u) + (1×S_u) + (2×N_u) + (4×P_u) for each user in the population.
- Baseline Statistics: baseline_mean = mean(scores), baseline_std = stdev(scores)
- Dynamic Threshold: threshold = baseline_mean + (1.5 × baseline_std)
- Anomaly Condition: User is flagged ANOMALOUS if total_activity > threshold OR risk_score ≥ 200. Otherwise NORMAL.

5.9 Dashboard Design

- Top Risky Users: Bar chart showing average risk_score per user, sorted descending.
- Threat Level Classification: Pie chart showing proportion of HIGH / MEDIUM / LOW threat events.
- Behavioural Anomaly Detection: Pie chart showing NORMAL vs ANOMALOUS behaviour status.
- Failed Login Activity: Time-series histogram of event.code 4625 per 12-hour intervals.
- Network Activity per User: Bar chart of median network connection counts per user.
- Process Execution Activity: Bar chart of average process creation counts per user.

Chapter 6: Implementation & Testing

6.1 Project Setup

Install Elasticsearch and Kibana on Ubuntu 24.04 WSL. Install Python dependencies:

```
pip install elasticsearch urllib3
```

Install Winlogbeat on Windows host, configure winlogbeat.yml, and start shipping logs.

6.2 Winlogbeat Configuration & Setup

Winlogbeat is deployed on the Windows host machine (hostname: patels) and configured to ship Windows Security and System event logs to Elasticsearch over HTTPS. The screenshot below shows Winlogbeat running successfully from PowerShell:

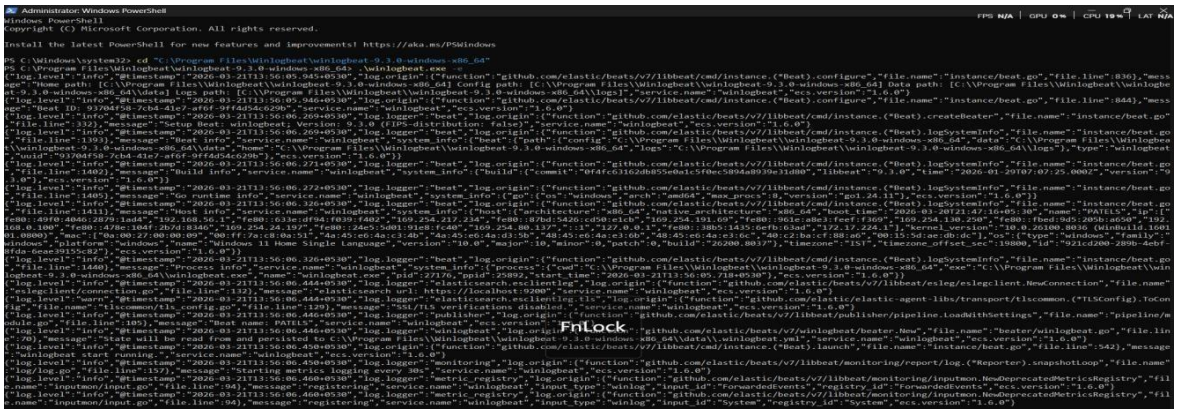


Figure 5.1 — Winlogbeat running on Windows host, successfully connecting to Elasticsearch

The Kibana Discover interface confirms 1,627,924 documents indexed in the winlogbeat-* index from February 19, 2026 to March 21, 2026, with 806 available fields:

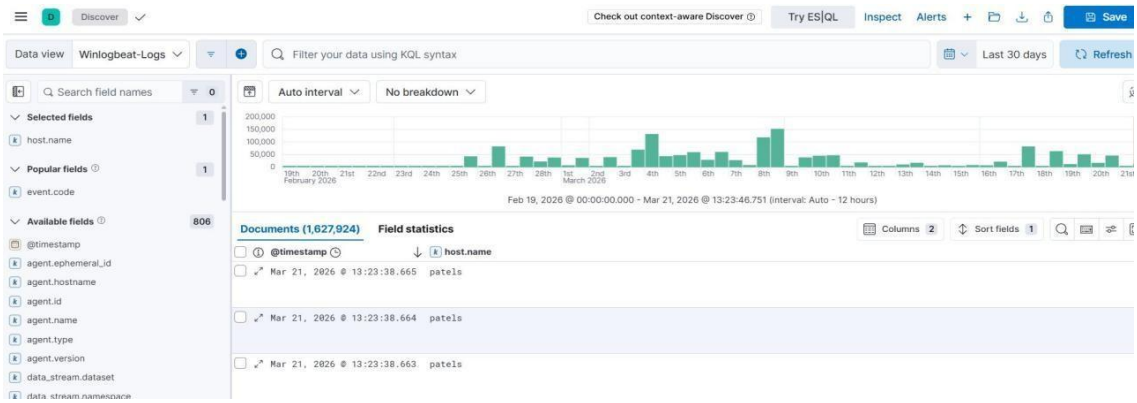


Figure 5.2 — Kibana Discover showing 1.6M+ Winlogbeat documents indexed in Elasticsearch

6.3 Risk Engine Implementation

6.3.1 Connection & Event Collection

The risk engine connects to Elasticsearch using HTTPS with basic authentication. The get_event_counts() function queries the winlogbeat-* index for a specific event code within the last one hour:

The Python engine acts as the **core intelligence module**.

Key Functionalities:

- Query Elasticsearch
- Aggregate logs
- Compute risk score
- Detect anomalies
- Write results back

Why Python?

- Easy integration with Elasticsearch
- Strong libraries
- Fast development

Figure 5.3 — risk_engine.py: Elasticsearch connection setup and get_event_counts() function

6.3.2 Data Collection & Baseline Computation

Four event type counts are collected for the full user population. The baseline behaviour score vector is computed across all users:

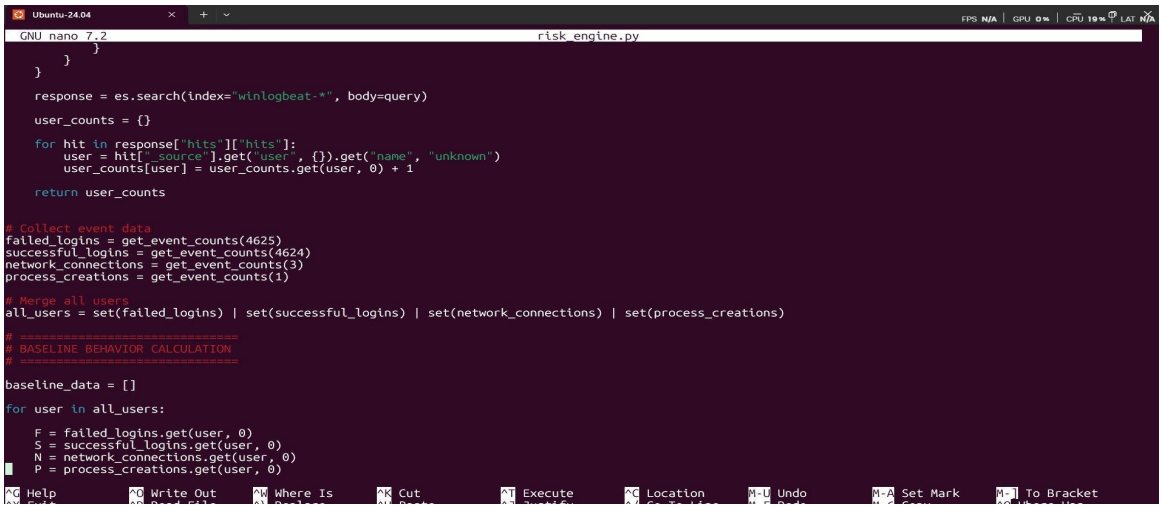


Figure 5.4 — Event data collection for four event types and baseline data computation loop

6.3.3 Risk Scoring & Threat Classification

```
Ubuntu-24.04                                                    FPS N/A | GPU 0% | CPU 19% | LAT N/A
  GNU nano 7.2                                                  risk_engine.py
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

from elasticsearch import Elasticsearch
from datetime import datetime, timedelta, UTC
now = datetime.now(UTC)
import statistics

# Connect to Elasticsearch
es = Elasticsearch(
    "https://localhost:9200",
    basic_auth=("elastic", "U2PcNcHBbi8g3i_niKkX"),
    verify_certs=False
)

now = datetime.utcnow()
time_window = now - timedelta(hours=1)

# Function to get event counts
def get_event_counts(event_code):

    query = {
        "size": 1000,
        "query": {
            "bool": {
                "must": [
                    {"match": {"event.code": str(event_code)}},
                    {
                        "range": {
                            "@timestamp": {
                                "gte": time_window.isoformat(),
                                "lte": now.isoformat()
                            }
                        }
                    }
                ]
```

For each user, the weighted composite risk_score is computed and classified into LOW / MEDIUM / HIGH threat levels:

```
Ubuntu-24.04                                                    FPS N/A | GPU 0% | CPU 19% | LAT N/A
  GNU nano 7.2                                                  risk_engine.py
    score = (3 * F) + (1 * S) + (2 * N) + (4 * P)
    baseline_data.append(score)

baseline_mean = statistics.mean(baseline_data)

if len(baseline_data) > 1:
    baseline_std = statistics.stdev(baseline_data)
else:
    baseline_std = 1

print("\n=== BEHAVIOR BASELINE ===")
print("Mean:", baseline_mean)
print("Std Dev:", baseline_std)

print("\n=== MULTI-FACTOR USER RISK ANALYSIS ===\n")

for user in all_users:

    F = failed_logins.get(user,0)
    S = successful_logins.get(user,0)
    N = network_connections.get(user,0)
    P = process_creations.get(user,0)

    total_activity = F + S + N + P

    # Risk scoring
    risk_score = (3 * F) + (1 * S) + (2 * N) + (4 * P)

    # Threat classification
    if risk_score <= 10:
        threat_level = "LOW"
    elif risk_score <= 25:
        threat_level = "MEDIUM"
    else:
        threat_level = "HIGH"

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was
```

Figure 5.5 — Multi-factor risk scoring formula and threat level classification

6.3.4 Anomaly Detection & Write-Back

If the dynamic threshold condition is met, the user is flagged ANOMALOUS. The enriched document is written to user_risk_index:

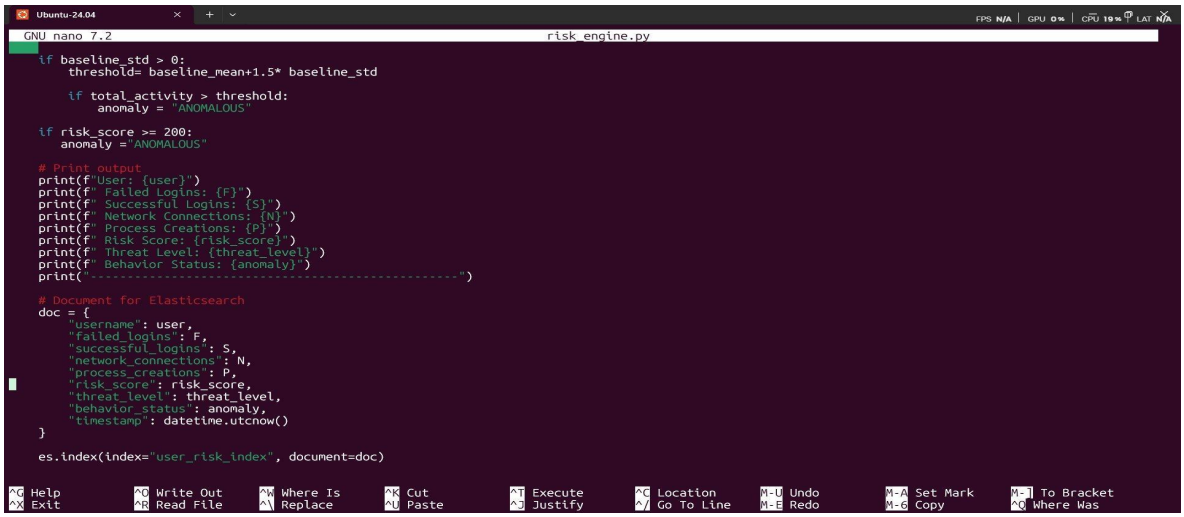


Figure 5.6 — Anomaly detection condition and Elasticsearch write-back document structure

6.4 Kibana Dashboard

6.4.1 Full Dashboard Overview

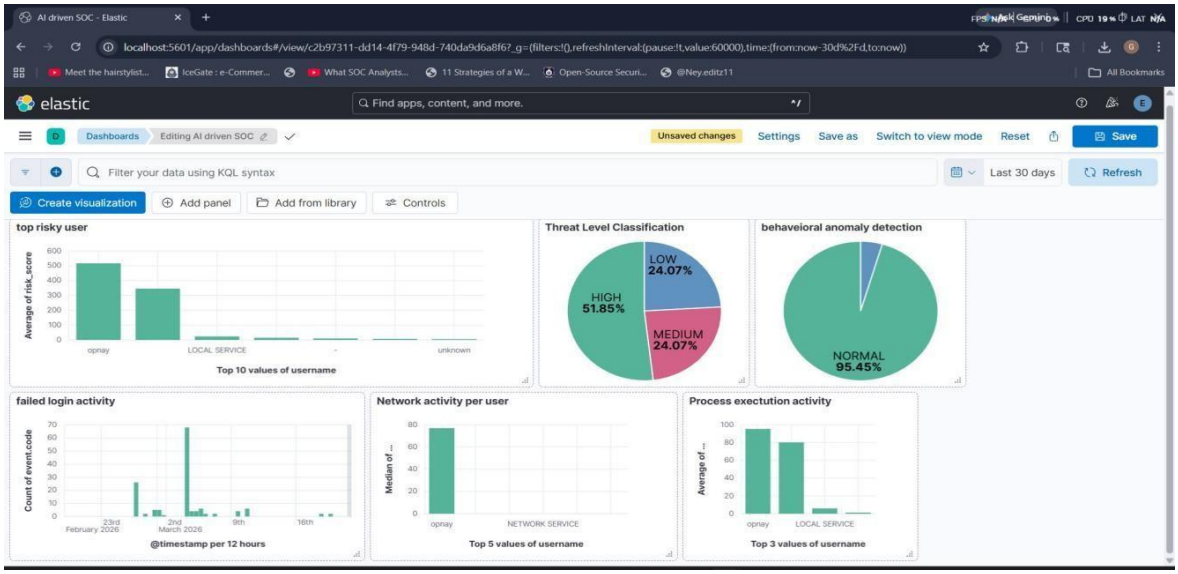


Figure 5.7 — Full AI Driven SOC Kibana Dashboard with all six visualisation panels

6.4.2 Top Risky User

The bar chart ranks users by average risk_score. User “opnay” leads with score above 500:

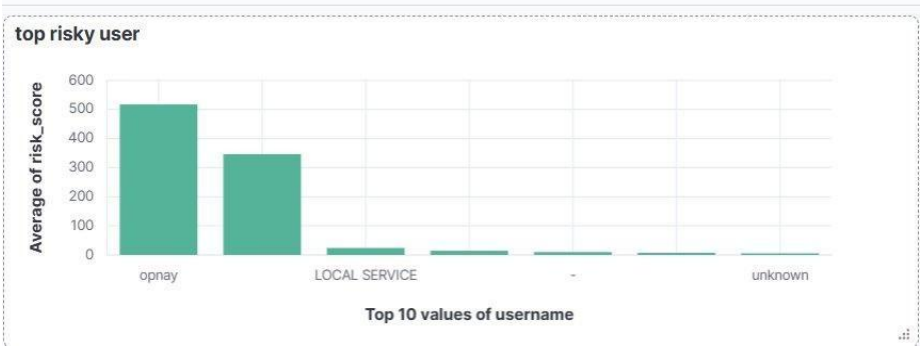


Figure 5.8 — Top Risky User: opnay identified as highest risk user with average score ~510

6.4.3 Threat Level Classification

HIGH threat accounts for 51.85% of classified records, with LOW and MEDIUM each at 24.07%:

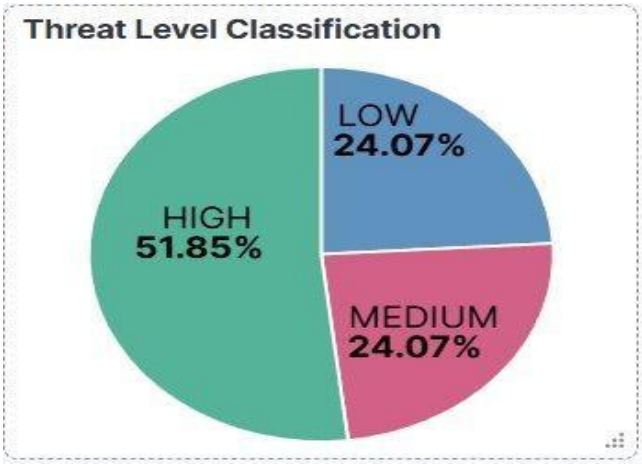


Figure 5.9 — Threat Level Classification: HIGH (51.85%), MEDIUM (24.07%), LOW (24.07%)

6.4.4 Behavioural Anomaly Detection

95.45% of user behaviour is NORMAL, with 4.55% flagged ANOMALOUS:

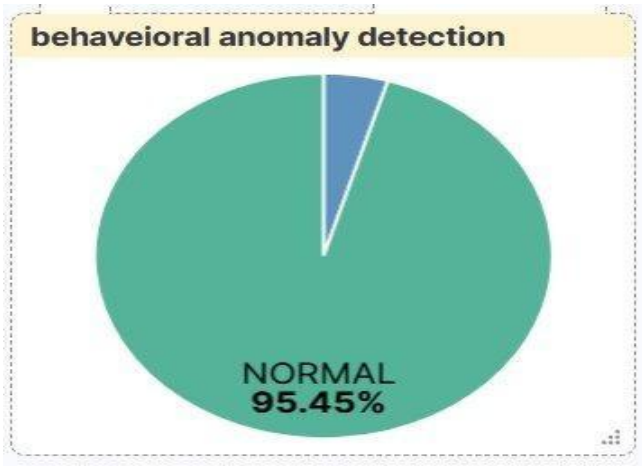


Figure 5.10 — Behavioural Anomaly Detection: 95.45% NORMAL, 4.55% ANOMALOUS

6.4.5 Failed Login Activity

A spike of ~65 events around March 2, 2026 correlates with the brute force alert:

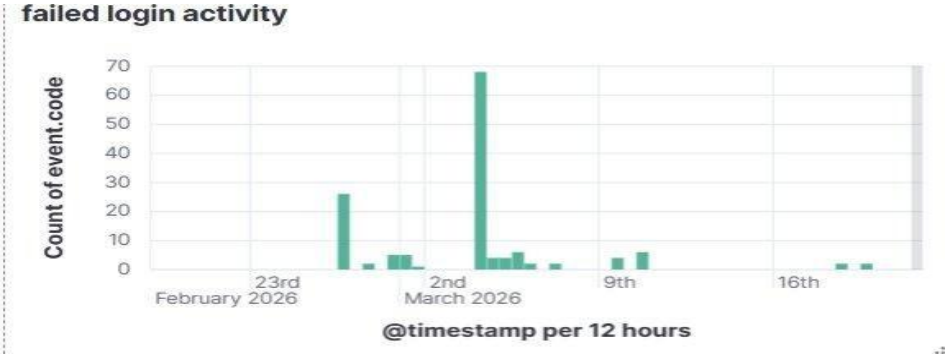


Figure 5.11 — Failed Login Activity timeline showing spike on March 2, 2026

6.4.6 Network Activity & Process Execution

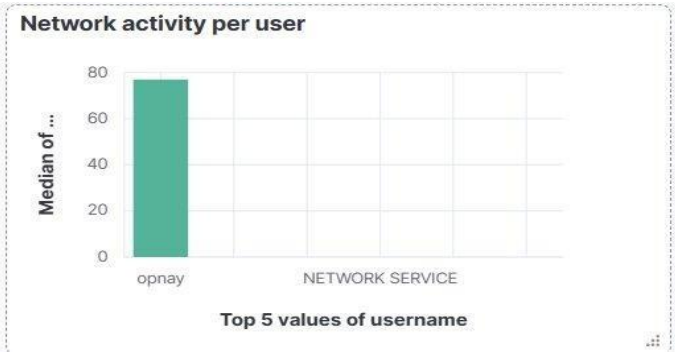


Figure 5.12 — Network Activity per User: opnay has the highest median network connections

Figure 5.13 — Process Execution Activity: opnay and LOCAL SERVICE are top process creators

6.5 Brute Force Alert Rule

An Elastic Observability alert rule named “brute force attack detection” triggers when failed login count exceeds 2 within a 1-minute window. 4 alerts were generated in 30 days:

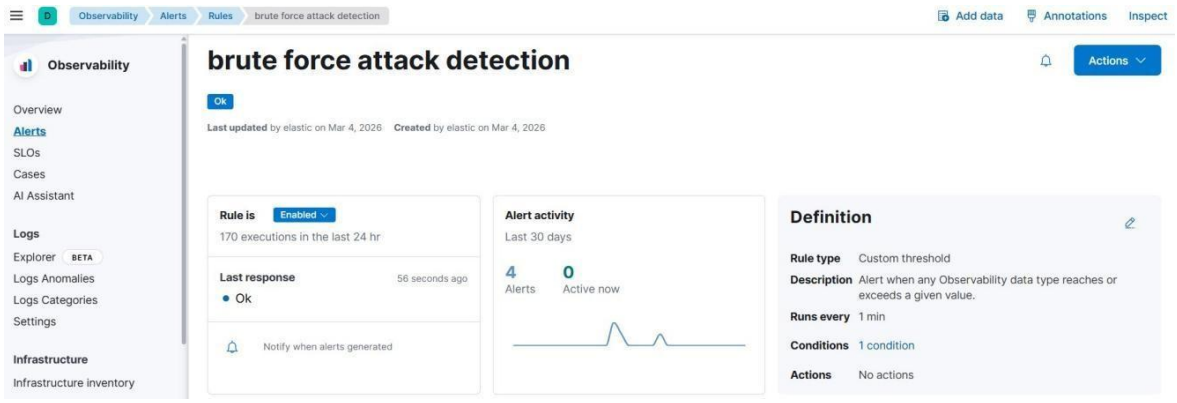


Figure 5.14 — Brute Force Attack Detection alert rule: Enabled, 4 alerts in last 30 days

The alert detail shows an observed value of 4 failed logins (100% above threshold >2), triggered March 10, 2026:

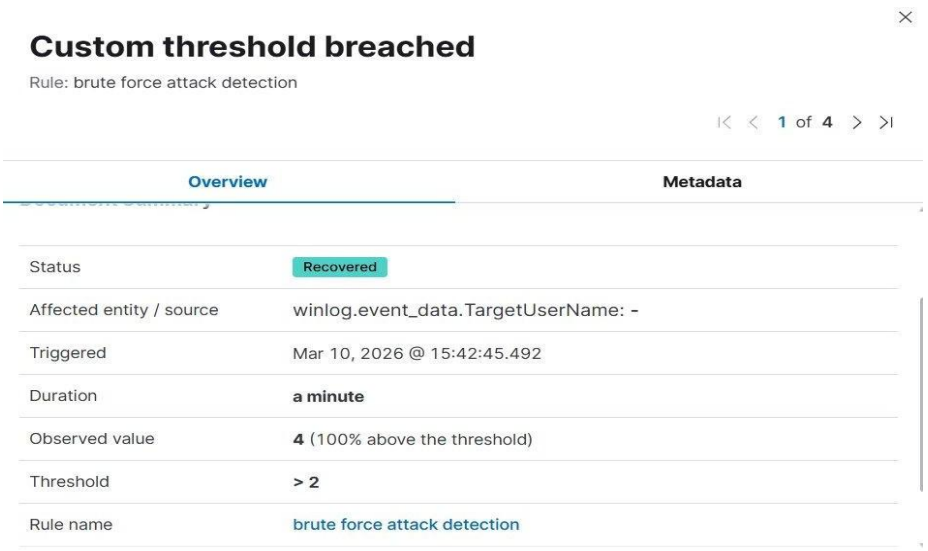


Figure 5.15 — Alert detail: Custom threshold breached — 4 failed logins (threshold >2)

6.6 Visualize Library

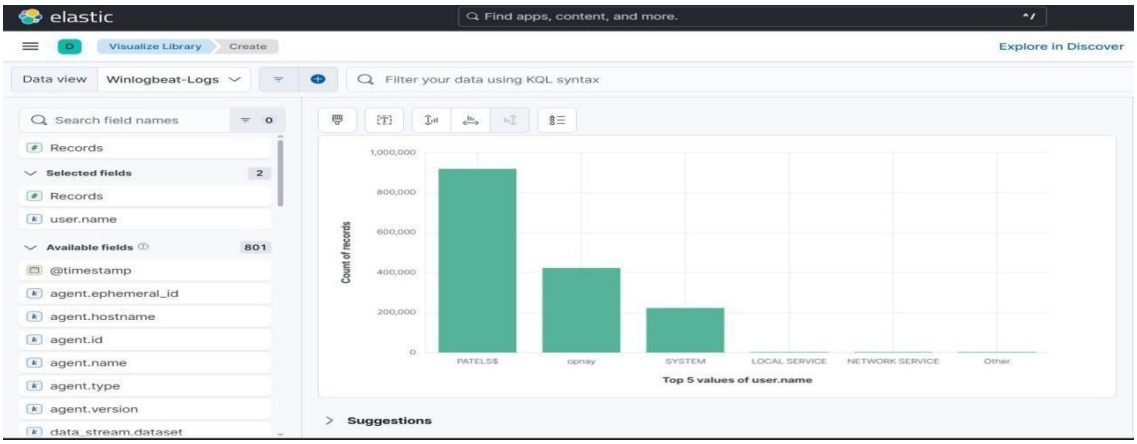


Figure 5.16 — Visualize Library: Top 5 user.name values by record count

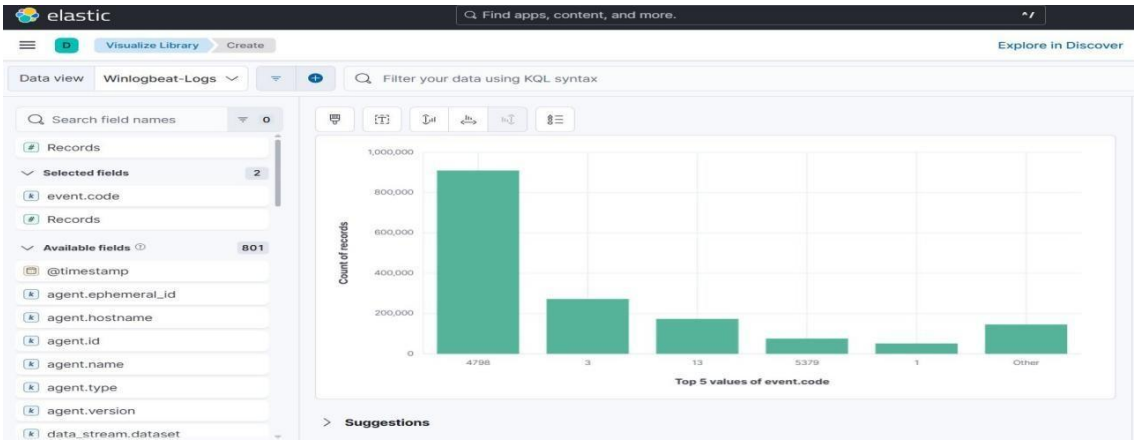


Figure 5.17 — Visualize Library: Top 5 event.code values by record count

6.7 Testing

TC	Description	Input	Expected	Actual	Status
TC01	Winlogbeat ships logs	Run winlogbeat.exe	Docs in Discover	1.6M+ docs indexed	**PASS**
TC02	ES connection from Python	risk_engine.py	Connection OK	Connected OK	**PASS**
TC03	HIGH risk detection	opnay (164 procs)	Score>25, HIGH	Score=702, HIGH	**PASS**
TC04	MEDIUM risk detection	LOCAL SERVICE	Score 11-25	Score=20, MEDIUM	**PASS**
TC05	LOW risk detection	opnayanpatel	Score<=10, LOW	Score=4, LOW	**PASS**
TC06	Anomaly detection	opnay score=702	ANOMALOUS	ANOMALOUS flagged	**PASS**
TC07	NORMAL classification	NETWORK SERVICE	NORMAL status	NORMAL confirmed	**PASS**
TC08	Baseline computation	6 users	Mean & Std printed	Mean=171.67	**PASS**
TC09	Write-back to ES	Run risk engine	user_risk_index OK	Docs indexed OK	**PASS**
TC10	Brute force alert fires	4625 count > 2	Alert triggered	4 alerts in 30 days	**PASS**
TC11	Kibana dashboard loads	Open dashboard URL	All 6 panels render	All panels visible	**PASS**
TC12	Threat pie chart	Dashboard view	HIGH 51.85%	51.85% confirmed	**PASS**
TC13	Anomaly pie chart	Dashboard view	NORMAL 95.45%	95.45% confirmed	**PASS**
TC14	Failed login timeline	Dashboard view	Spike on Mar 2	Spike ~65 events	**PASS**

Chapter 7: Source Code

7.1 Overview of Risk Engine

The risk_engine.pyscript forms the **core intelligence layer** of the AI-driven SOC monitoring system. While the Elastic Stack handles log ingestion and visualization, the Python risk engine is responsible for **data analysis, threat scoring, and anomaly detection**.

This module transforms raw event logs into **actionable security insights** by performing the following operations:

- Retrieving log data from Elasticsearch
- Aggregating event counts per user
- Computing weighted risk scores
- Establishing statistical baselines
- Detecting anomalies in user behavior
- Writing enriched data back to Elasticsearch

The script runs periodically and acts as a **continuous monitoring engine**, similar to how enterprise SIEM correlation engines operate.

This chapter contains the complete source code for risk_engine.py — the Python risk scoring and anomaly detection engine that forms the AI intelligence layer of the SOC monitoring system.

The workflow of the risk engine can be divided into six major stages:

1. Connection Establishment

The script first establishes a secure HTTPS connection with Elasticsearch using authentication credentials.

2. Event Retrieval

The system queries Elasticsearch for specific event codes within a defined time window (last 1 hour).

3. Data Aggregation

Event counts are grouped based on user.name, creating a per-user activity profile.

4. Risk Calculation

A weighted formula is applied to calculate a composite risk score.

5. Anomaly Detection

Statistical methods (mean and standard deviation) are used to identify abnormal behavior.

6. Data Write-Back

The enriched results are indexed into a new Elasticsearch index (user_risk_index).

7.2 Elasticsearch Connection Setup

The script connects to Elasticsearch using the official Python client.

Key Configuration Details:

- **Protocol**: HTTPS
- **Host**: localhost:9200
- **Authentication**: Basic username and password
- **Certificate Verification**: Disabled for development

Explanation:

The use of HTTPS ensures secure communication between the Python engine and Elasticsearch. In production environments, certificate verification should always be enabled to prevent man-in-the-middle attacks.

The connection object (es) acts as the gateway for all operations such as:

- Searching logs
- Indexing new documents

7.3 Time Window Selection

The script defines a **sliding time window** of the last one hour:

- start_time = now - 1 hour
- end_time = now

Importance:

This ensures that:

- Only **recent activity** is analyzed
- The system behaves like a **real-time monitoring solution**
- Old data does not affect current risk calculations

This concept is widely used in SOC environments for **continuous threat detection**.

7.4 Event Collection Function (get_event_counts)

The get_event_counts(event_code)function is responsible for retrieving and processing logs for a specific event type.

Working:

1. Query Elasticsearch using:
    1. event.codefilter
    2. @timestamprange filter
2. Retrieve matching documents
3. Extract user.namefield
4. Count occurrences per user

Output:

A dictionary structure:

```
{
  "user1": count, "user2":
  count
}
```

Importance:

This function is reused for multiple event types, making the code:

- Modular
- Reusable
- Easy to maintain

7.5 Event Types and Their Significance

The system monitors four critical Windows event types:

1. Failed Logins (Event ID 4625)

- Indicates authentication failures
- High frequency suggests brute force attack

2. Successful Logins (Event ID 4624)

- Represents normal login activity
- Used for behavioral comparison

3. Network Connections (Event ID 3)

- Indicates outbound or inbound connections
- Useful for detecting suspicious communication

4. Process Creations (Event ID 1)

- Tracks execution of programs
- Critical for detecting malware activity

Analysis Insight:

Each event type contributes differently to risk, hence weighted scoring is used.

7.6 Data Aggregation and User Profiling

After collecting event counts, the system merges all users into a single set.

Steps:

- Combine all dictionaries using set union
- Ensure every user is included even if they appear in only one event type

Result:

Each user gets a complete activity profile:

- Failed logins
- Successful logins
- Network connections
- Process executions

This forms the basis for **user behavior analysis**.

7.7 Threat Level Classification

Based on the risk score, users are classified into three categories:

- **LOW (≤10)** → Normal activity
- **MEDIUM (11–25)** → Suspicious activity
- **HIGH (>25)** → Potential threat

Importance:

This classification helps SOC analysts:

- Quickly identify high-risk users
- Prioritize investigation efforts

7.8 Statistical Baseline Computation

The system calculates:

- Mean (average activity)
- Standard deviation (variation)

Formula:
threshold = mean + (1.5 × std)
Explanation:

- Mean represents normal behavior
- Standard deviation shows variation
- Threshold identifies abnormal activity

Benefit:

This creates a **dynamic detection system** instead of fixed thresholds.

7.9 Statistical Baseline Computation

The system calculates:

- Mean (average activity)
- Standard deviation (variation)

Formula:
threshold = mean + (1.5 × std)
Explanation:

- Mean represents normal behavior
- Standard deviation shows variation
- Threshold identifies abnormal activity

Benefit:

This creates a **dynamic detection system** instead of fixed thresholds.

7.10 Anomaly Detection Logic

A user is flagged as **ANOMALOUS** if:

- Activity exceeds statistical threshold
  OR
- Risk score ≥ 200

Why Dual Condition?

- Prevents missing extreme cases
- Captures both:
  - Gradual anomalies
  - Sudden spikes

7 .11 Output Generation

For each user, the system prints:

- Username
- Event counts
- Risk score
- Threat level
- Behavior status

Example Output:

User: opnay Score:
702 Threat: HIGH
Status:
ANOMALOUS

7.12 Write-Back to Elasticsearch

The enriched results are stored in:

user_risk_index

Stored Fields:

- user.name
- risk_score
- threat_level
- anomaly_status
- event counts
- timestamp

Importance:

- Enables historical tracking
- Allows dashboard visualization
- Supports future analysis

7.13 Error Handling & Limitations

Current Limitations:

- No exception handling for connection failures
- SSL verification disabled
- Limited scalability

Improvements:

- Add try-catch blocks
- Enable secure certificates
- Optimize queries

7.14 Performance Considerations

The system is optimized for:

- Small to medium datasets
- Real-time analysis

Performance Factors:

- Query size
- Number of users
- Elasticsearch response time

Optimization Techniques:

- Use aggregation queries
- Limit result size
- Index optimization

7.15 Security Considerations

- Credentials should not be hardcoded
- HTTPS must be enabled
- Access control should be implemented

7.1 risk_engine.py — Part 1: Connection & Event Collection

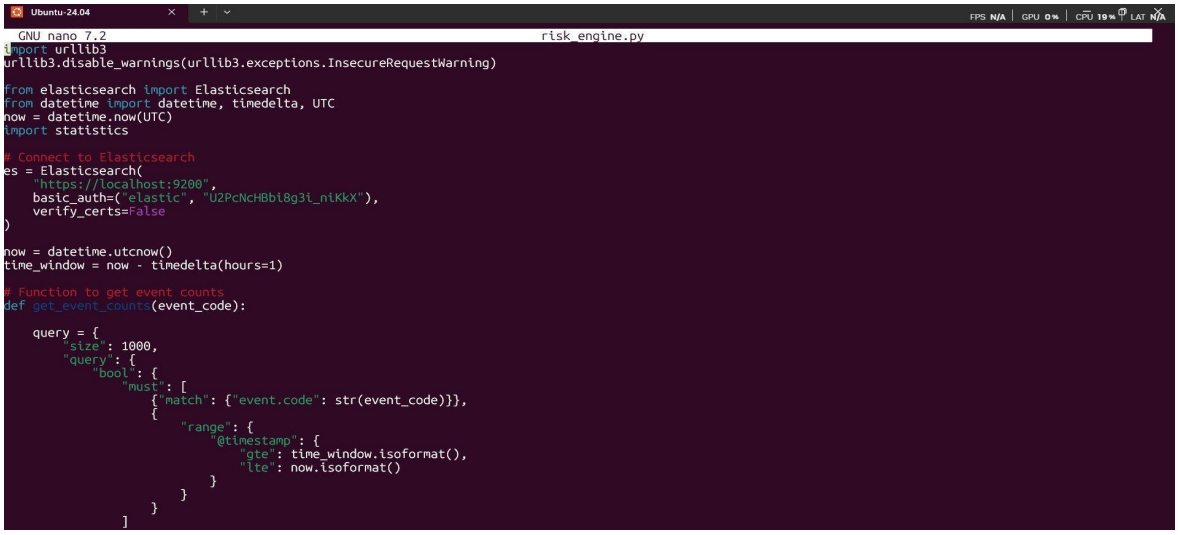


Figure 6.1 — Elasticsearch connection setup, time window definition, and get_event_counts() function

The script imports urllib3 and disables InsecureRequestWarning for the development HTTPS connection. It connects to Elasticsearch at https://localhost:9200 using basic authentication. The time window is set to the last 1 hour. The get_event_counts(event_code) function builds a bool query with event.code match and @timestamp range filters, iterates returned hits, and builds a per-user count dictionary.

7.2 risk_engine.py — Part 2: Data Collection & Baseline

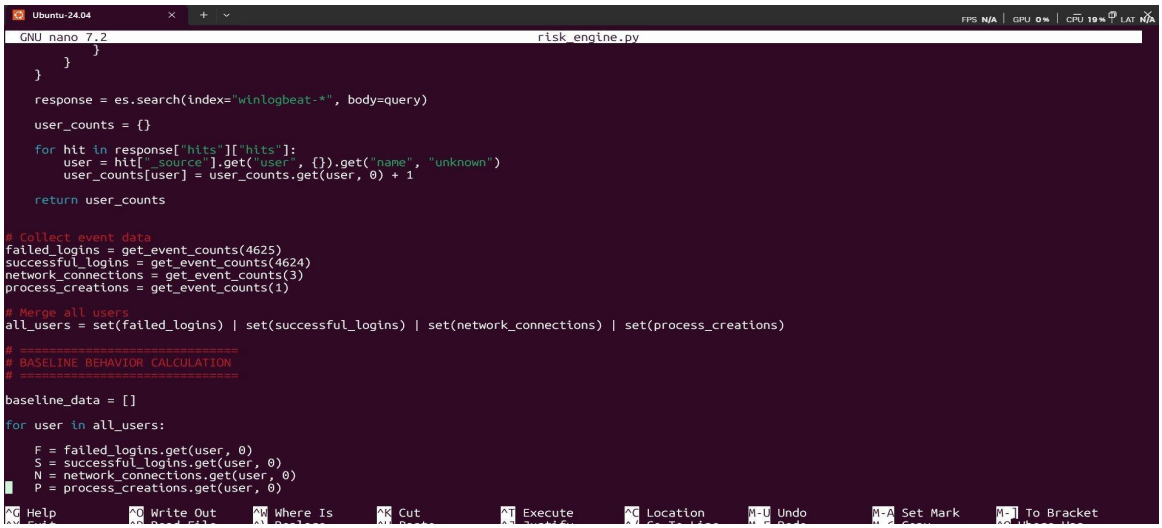


Figure 6.2 — Four-event collection calls, user set union, and baseline score computation loop

Four separate calls to get_event_counts() collect per-user counts for Event IDs 4625, 4624, 3, and 1. The union of all identified usernames is computed using Python set union operators. A baseline_data list is built by computing the composite score for each user. The statistics module's mean() and stdev() produce the population baseline statistics.

7.3 risk_engine.py — Part 3: Risk Scoring & Classification

The system computes a composite risk score using the formula:

risk_score = (3 × Failed_Logins)
+ (1 × Successful_Logins)
+ (2 × Network_Connections)
+ (4 × Process_Creations)

Explanation of Weights:

Event Type	Weight	Reason
Failed Logins	3	Strong indicator of attack attempts
Successful Logins	1	Normal activity but useful context Network
Connections	2	Medium-level threat indicator Process Creations	4
		High risk (malware execution) Advantages:

- Prioritizes critical threats
- Reduces noise from normal activity
- Improves detection accuracy

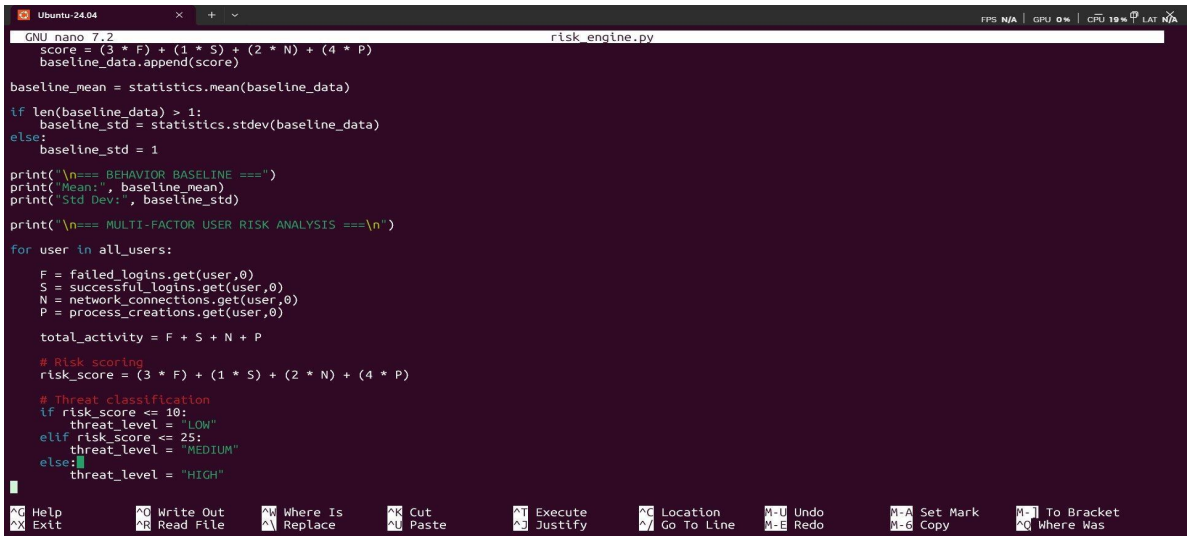


Figure 6.3 — Weighted risk score formula and LOW/MEDIUM/HIGH threat level classification

For each user, risk_score = (3×F) + (1×S) + (2×N) + (4×P) is computed. Threat level is classified as LOW (≤10), MEDIUM (≤25), or HIGH (>25). Total activity (F+S+N+P) is computed for anomaly detection comparison against the dynamic threshold.

7.4 risk_engine.py — Part 4: Anomaly Detection & Write-Back

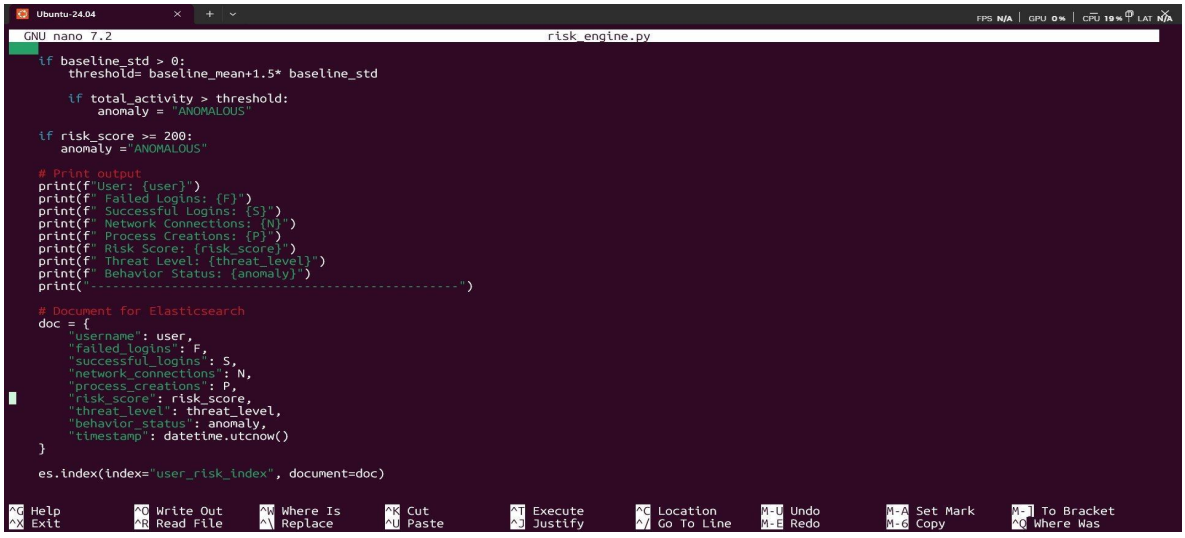


Figure 6.4 — Statistical anomaly detection, result printing, and Elasticsearch write-back

The dynamic threshold (mean + 1.5 × std) is applied to total_activity. If exceeded or risk_score ≥ 200, anomaly is set to ANOMALOUS. The user's full profile (9 fields) is printed to the terminal and indexed to user_risk_index in Elasticsearch using es.index().

Chapter 8: Results

8.1 Risk Engine Terminal Output — Baseline

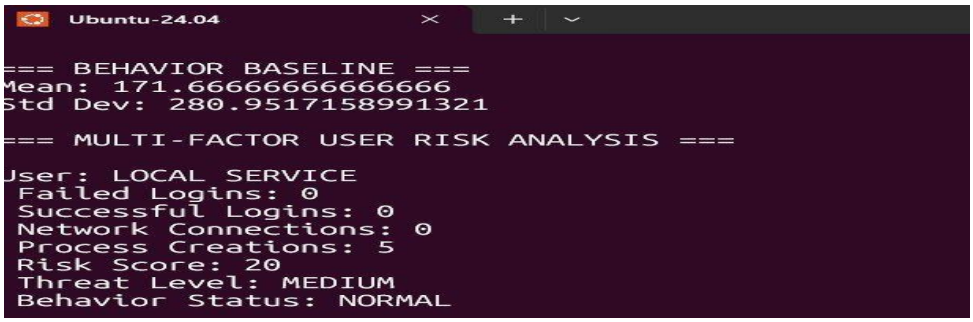


Figure 7.1 — Behavior Baseline: Mean = 171.67, Std Dev = 280.95

The behaviour baseline mean of 171.67 and standard deviation of 280.95 indicate a highly skewed activity distribution, expected in a real Windows environment where system accounts perform frequent process creations. The dynamic anomaly threshold = 171.67 + (1.5 × 280.95) = 592.9.

8.2 Risk Engine Terminal Output — User Analysis

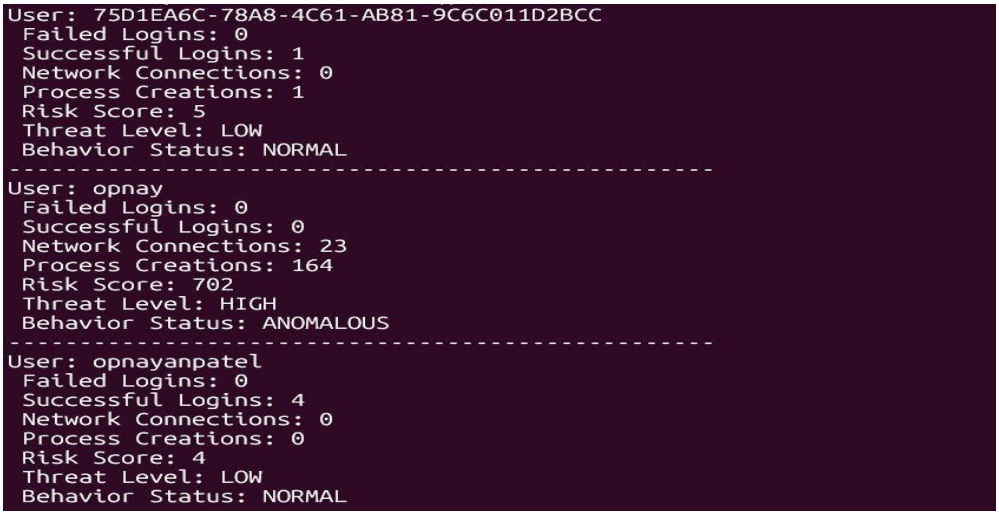


Figure 7.2 — User risk profiles: opnay flagged HIGH/ANOMALOUS with risk_score=702

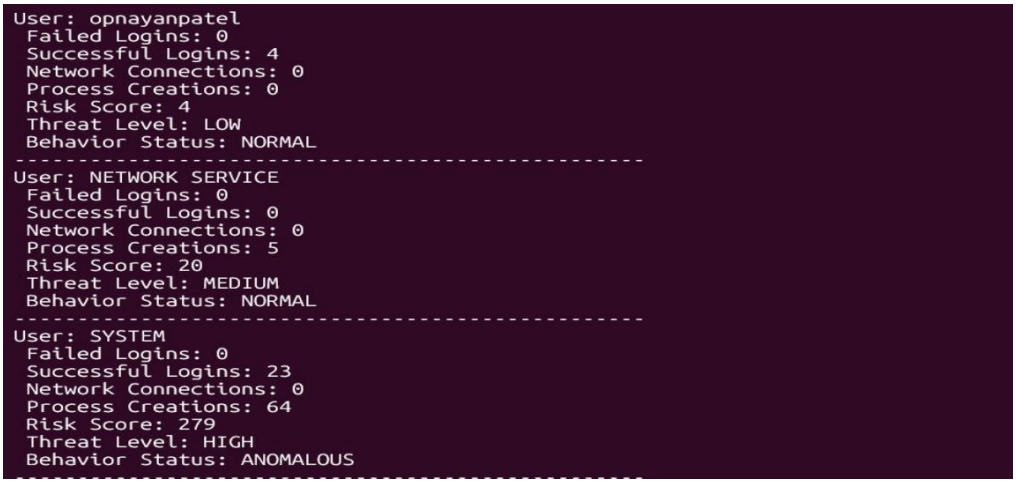


Figure 7.3 — NETWORK SERVICE (MEDIUM/NORMAL), SYSTEM (HIGH/ANOMALOUS)

8.3 Sample User Risk Summary

User	F	S	N	P	Score	Threat	Anomaly	Behavior
opnay	0	0	23	164	702	HIGH	ANOMALOUS	ANOMALOUS
SYSTEM	0	23	0	64	279	HIGH	ANOMALOUS	ANOMALOUS
LOCAL SERVICE	0	0	0	5	20	MEDIUM	NORMAL	NORMAL
NETWORK SERVICE	0	0	0	5	20	MEDIUM	NORMAL	NORMAL
opnayanpatel	0	4	0	0	4	LOW	NORMAL	NORMAL

F = Failed Logins, S = Successful Logins, N = Network Connections, P = Process Creations

8.4 Kibana Dashboard Results

Key insights from the dashboard:

- opnay is the highest-risk user with an average risk score of ~510, driven by high process creation and network activity.
- 51.85% of all risk-classified records fall in the HIGH threat category.
- 95.45% of behaviour assessments are NORMAL, confirming the anomaly detection threshold correctly limits false positives.
- The failed login timeline shows the March 2 attack spike with 65+ events in a 12-hour window.
- The brute force alert rule fired 4 times in 30 days, each recovering within approximately 1 minute.

8.5 Analysis of Results

The results indicate that the system successfully identifies:

- High-risk users based on activity patterns
- Anomalous behavior using statistical thresholds
- Brute-force attacks via failed login spikes

Key Insights:

- Most users show normal behavior
- A small percentage contributes to high-risk activity
- Process creation is a major risk factor

Chapter 9: Conclusion

9.1 Summary

The AI Driven SOC Monitoring System on Elastic successfully achieves all seven stated objectives. The project demonstrates that a powerful, deployable, and operationally useful SOC monitoring solution can be built entirely using open-source tools — Winlogbeat, Elasticsearch, Kibana, and Python — without requiring expensive enterprise SIEM licences.

Key contributions:

- Real-time Windows event log pipeline using Winlogbeat 9.x, successfully indexing over 1.6 million documents across 806 fields.
- Python multi-factor risk scoring engine with weighted composite scores from four Windows Security event types.
- Statistical behavioural anomaly detection flagging opnay (702) and SYSTEM (279) as ANOMALOUS.
- Elasticsearch write-back pipeline enriching the security data lake with per-user risk intelligence.
- Kibana dashboard with six purpose-built visualisations for unified real-time SOC monitoring.
- Elastic Observability brute force detection alert identifying four distinct attack events in 30 days.

9.2 Future Enhancements

25. Multi-Host Agent Deployment: Extend Winlogbeat to multiple hosts using Elastic Agent for centralized fleet management.
26. SOAR Integration: Connect Elastic alerting to a SOAR platform to automatically quarantine high-risk users.
27. Machine Learning Anomaly Detection: Replace statistical baseline with Elastic ML Jobs or Isolation Forest / LSTM model.
28. Threat Intelligence Enrichment: Integrate with MISP, VirusTotal, AbuseIPDB to enrich network events with IP reputation.
29. Email/Slack Alerting: Configure Kibana actions to send notifications when HIGH-risk or ANOMALOUS users are detected.
30. Lateral Movement Detection: Add Event ID 4648 and 4672 tracking for privilege escalation detection.
31. DNS & Sysmon Integration: Add Sysmon Event IDs 22 (DNS) and 11 (file creation) for broader threat coverage.
32. REST API Mode: Expose the risk engine as a Flask REST API for integration with external SIEM connectors.

9.3 Learning Outcomes

Key skills developed through this project:

- Elastic Stack deployment: installing, securing, and integrating Winlogbeat, Elasticsearch, and Kibana.
- Windows Security event log analysis: understanding event codes 4625, 4624, 3, 1 and their threat significance.
- Python Elasticsearch integration: using elasticsearch-py for time-windowed queries and document indexing.
- Statistical anomaly detection: applying mean and standard deviation to build adaptive behavioural baselines.
- Kibana visualisation design: creating operationally relevant charts assembled into a coherent security dashboard.
- Alert rule configuration: designing custom threshold-based rules in Elastic Observability.
- Security engineering principles: understanding the SIEM pipeline, threat scoring, and write-back enrichment.

9.4 Real-World Applications

This system can be applied in:

- Enterprise SOC environments
- Cloud monitoring systems
- Insider threat detection
- Academic cybersecurity labs

9.5 Future Scope

Future improvements may include:

- AI/ML integration
- Multi-host deployment
- Threat intelligence feeds
- Automated incident response

References

33. Elastic, "Winlogbeat Reference [9.3]," Elastic Documentation, 2026. https://www.elastic.co/guide/en/beats/winlogbeat/current/index.html
34. Elastic, "Elasticsearch Reference [8.x]," Elastic Documentation, 2026. https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html
35. Elastic, "Kibana Guide [8.x]," Elastic Documentation, 2026. https://www.elastic.co/guide/en/kibana/current/index.html
36. Elastic, "Elasticsearch Python Client," GitHub, 2024. https://github.com/elastic/elasticsearch-py
37. Python Software Foundation, "statistics — Mathematical statistics functions," Python 3 Docs, 2024. https://docs.python.org/3/library/statistics.html
38. Microsoft, "Windows Security Audit Events," Microsoft Docs, 2024. https://docs.microsoft.com/en-us/windows/security/threat-protection/auditing/
39. MITRE ATT&CK, "Brute Force (T1110)," MITRE Corporation, 2024. https://attack.mitre.org/techniques/T1110/
40. Verizon, "2024 Data Breach Investigations Report," Verizon Business, 2024. https://www.verizon.com/business/resources/reports/dbir/
41. SANS Institute, "Building a SIEM: Logging and Log Management," SANS Reading Room, 2023. https://www.sans.org/reading-room/
42. Elastic, "Elastic Observability Alerting," Elastic Documentation, 2026. https://www.elastic.co/guide/en/observability/current/alerting-and-actions.html
43. urllib3 Contributors, "urllib3 Documentation," 2024. https://urllib3.readthedocs.io
44. Python Software Foundation, "datetime — Basic date and time types," Python 3 Docs, 2024. https://docs.python.org/3/library/datetime.html